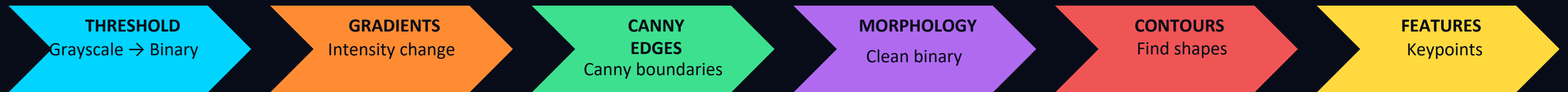


Edge Detection, Contours & Feature Extraction

Extracting structure, boundaries, and matchable keypoints from raw pixel data



Today's Journey:



Image Thresholding

Converting grayscale images to binary — the essential first step for shape analysis

How It Works — Visual Example

Each pixel has an intensity from 0 (black) to 255 (white).

Thresholding compares every pixel to a threshold value T . If $\text{pixel} > T \rightarrow \text{white (foreground)}$. If $\text{pixel} \leq T \rightarrow \text{black (background)}$.

Grayscale Input:

40	80	200	180
60	220	240	30
190	210	50	70

$T = 127$



Binary Output:

0	0	255	255
0	255	255	0
255	255	0	0

Result: every pixel is now either 0 or 255, making it easy to find shapes, separate foreground from background, and feed into contour detection.

Code Quick Reference

```
# Convert to grayscale first
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Simple binary thresholding
_, binary = cv2.threshold(gray, 127, 255,
                          cv2.THRESH_BINARY)
```

Three Thresholding Methods

Simple Binary

You choose T manually

Compare every pixel to a single fixed threshold.

- If $\text{pixel} > T \rightarrow \text{set to 255 (white)}$
- If $\text{pixel} \leq T \rightarrow \text{set to 0 (black)}$

Best for: uniform lighting, controlled environments. Fails with shadows.

Otsu's Automatic

Algorithm finds optimal T

Analyzes the image histogram to find the threshold that minimizes intra-class variance (variance within foreground + background groups). No manual tuning needed. Works best when histogram has two clear peaks.
Code: `cv2.threshold(gray, 0, 255, THRESH_BINARY + THRESH_OTSU)`

Adaptive

Different T per region

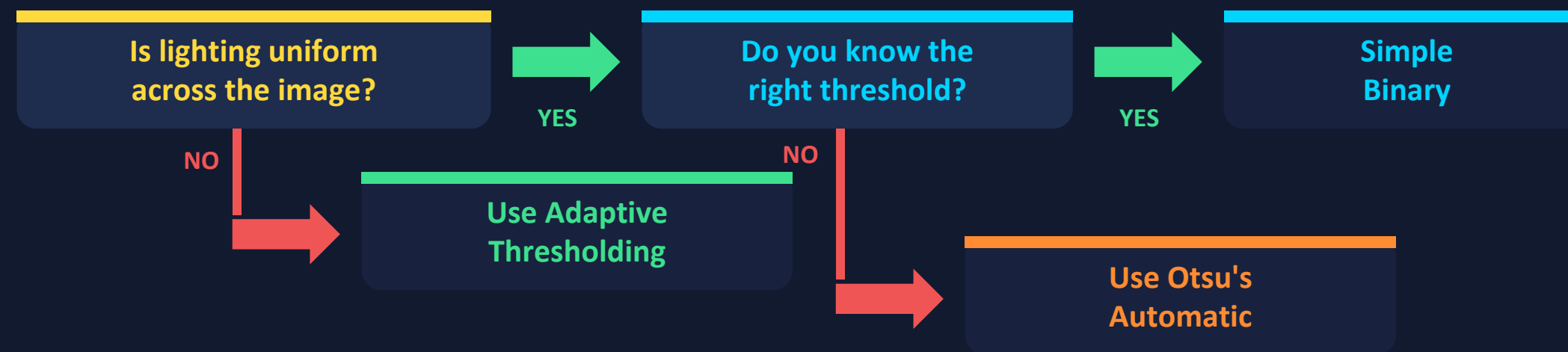
Calculates a separate threshold for each small neighborhood of pixels. Two variants: Mean (average of neighbors) and Gaussian (weighted average). Handles shadows, uneven lighting, and varying illumination across the image.
Code: `cv2.adaptiveThreshold(gray, 255, ADAPTIVE_THRESH_GAUSSIAN_C, ...)`

1b

Which Thresholding Method Should You Use?

A decision guide based on your image conditions

Decision Flowchart: Which Thresholding Method?



Code Quick Reference

```
# Simple Binary
cv2.threshold(gray, 127, 255, THRESH_BINARY)

# Otsu's Automatic
cv2.threshold(gray, 0, 255, THRESH_BINARY+THRESH_OTSU)

# Adaptive Gaussian
cv2.adaptiveThreshold(gray, 255,
    ADAPTIVE_THRESH_GAUSSIAN_C, THRESH_BINARY, 11, 2)
```

Simple Binary

Use when you have consistent lighting and can visually determine the threshold.

Example: document scanning with controlled lighting setup.

Otsu's Automatic

Use when lighting is even but you don't want to guess the threshold.

Example: separating cells from background in microscope images.

Adaptive

Use when lighting varies across the image (shadows, reflections).

Example: reading text on a page photographed with phone flash.

Image Gradients — Measuring Intensity Change

The mathematical foundation of all edge detection algorithms

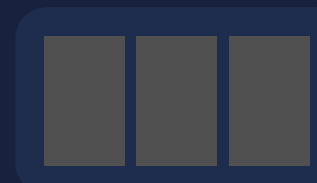
What is an Image Gradient?

A gradient measures how quickly pixel intensity changes.

- In a flat region: all pixels are similar → gradient is near zero.
- At an edge: pixels change sharply (dark → light) → high gradient.
- At a corner: intensity changes in multiple directions → very high gradient.

Think of it like slope on a terrain map — steep slope = edge, flat = no feature.

Three Types of Regions



Flat

Gradient: ≈ 0

No intensity change anywhere



Edge

Gradient: Medium

Change in one direction only



Corner

Gradient: High

Change in all directions

Sobel X Kernel

Detects vertical edges by measuring horizontal (left↔right) intensity change.

-1	0	+1	← Left pixels get negative weights
-2	0	+2	
-1	0	+1	
			→ Right pixels get positive weights

Sobel Y Kernel

Detects horizontal edges by measuring vertical (top↔bottom) intensity change.

-1	-2	-1	↑ Top row negative
0	0	0	
+1	+2	+1	↓ Bottom row positive

Combining Gradients

Magnitude (edge strength):

$$M = \sqrt{G_x^2 + G_y^2}$$

Direction (edge angle):

$$\theta = \arctan(G_y / G_x)$$

Code

```
Gx = cv2.Sobel(gray, CV_64F, 1, 0)
Gy = cv2.Sobel(gray, CV_64F, 0, 1)
mag = np.sqrt(Gx**2 + Gy**2)
lap = cv2.Laplacian(gray, CV_64F)
```

Gradient Operators — Compared

Sobel vs Scharr vs Laplacian — when to use which

Sobel

1st Order Derivative

- Most common gradient operator
- Computes X and Y separately
- Moderate noise sensitivity
- Use ksize=3 (default) or ksize=5
- Good balance of speed & accuracy

```
cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

When to use:

General-purpose edge detection.
Default choice for most projects.

Scharr

Enhanced 1st Derivative

- Better rotational symmetry than Sobel
- More accurate for 3×3 kernel size
- Lower noise sensitivity
- Only works with ksize=3
- Weights: 3,10,3 vs Sobel's 1,2,1

```
cv2.Scharr(gray, cv2.CV_64F, 1, 0)
```

When to use:

When you need more accurate gradients
at the cost of only supporting 3×3.

Laplacian

2nd Order Derivative

- Detects edges in ALL directions at once
- No need for separate X and Y
- Computes the sum of second derivatives
- Very sensitive to noise (blur first!)
- Finds both edges and blobs

```
cv2.Laplacian(gray, cv2.CV_64F)
```

When to use:

Quick overview of all edges.
Always blur image first.

3

Canny Edge Detection — The Gold Standard

John Canny, 1986 — Optimal: low error rate, good localization, thin single-pixel edges

1

GAUSSIAN SMOOTHING

5×5 Gaussian kernel removes sensor noise. Without this, noise creates thousands of false edges.

Input:

Noisy grayscale image

2

GRADIENT COMPUTATION

Sobel X & Y compute magnitude and angle at every pixel. Strong gradient = likely edge.

Output:

Gradient magnitude + direction map

3

NON-MAX SUPPRESSION

For each pixel, check if it's the local max along gradient direction. Thick → thin edges. Single-pixel lines.

Effect:

Thick edge regions become thin lines

4

HYSTERESIS THRESHOLDING

Two thresholds:
> high = sure edge
< low = not edge
between = edge only if connected to sure.

Key:

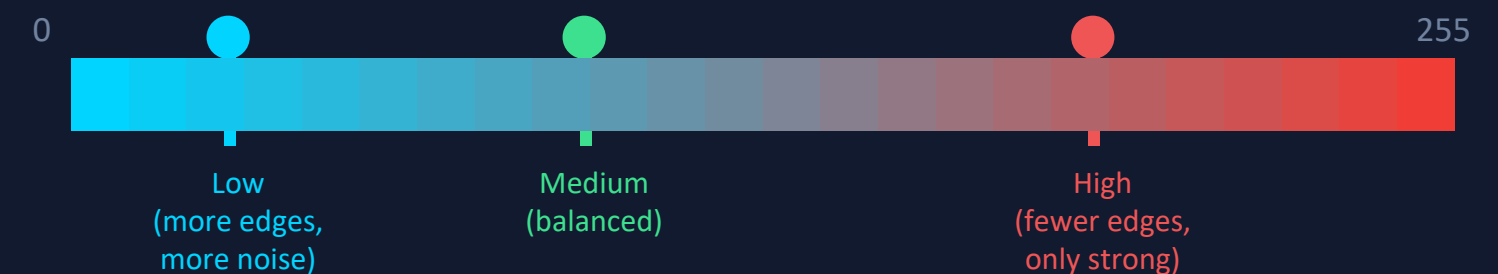
Keeps weak edges that connect to strong

Code

```
# Always blur before Canny!
blurred = cv2.GaussianBlur(gray, (5,5), 0)
edges = cv2.Canny(blurred, 50, 150)

# Auto-Canny (adaptive thresholds):
median = np.median(image)
lower = int(max(0, (1 - 0.33) * median))
upper = int(min(255, (1 + 0.33) * median))
edges = cv2.Canny(image, lower, upper)
```

Threshold Selection Guide



Canny recommended ratio: high = 2× to 3× low | Common pairs: (50,150) (100,200) (30,90)

Morphological Operations

Sculpting binary images — remove noise, fill holes, refine shapes before analysis

EROSION

Shrink foreground

Slides a kernel across the image. A pixel stays white only if ALL pixels under the kernel are white.

Effect: shrinks objects, removes small noise dots, separates touching objects.



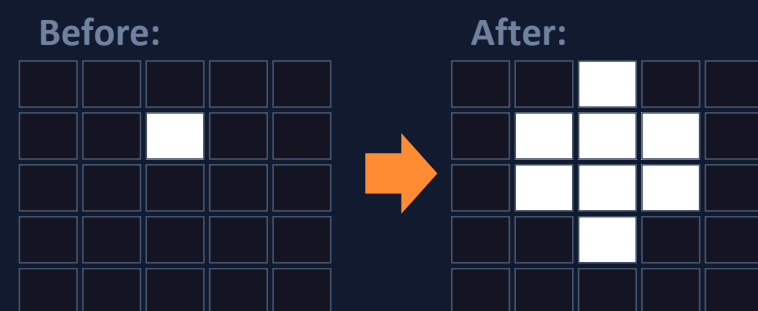
```
cv2.erode(img, kernel, iterations=1)
```

DILATION

Expand foreground

Slides a kernel across the image. A pixel becomes white if ANY pixel under the kernel is white.

Effect: expands objects, fills small holes, connects nearby components.



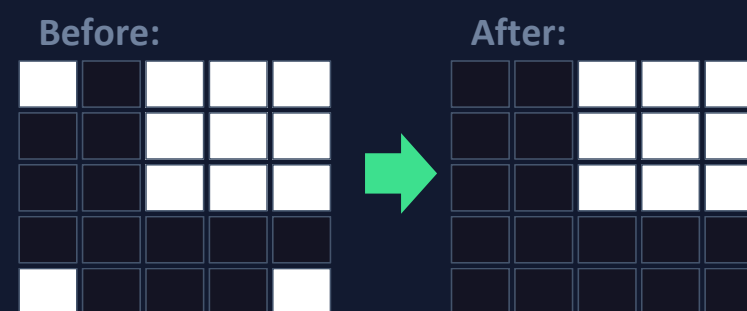
```
cv2.dilate(img, kernel, iterations=1)
```

OPENING

Erode → Dilate

First erodes (removes noise), then dilates (restores size).

Effect: noise specks are removed but the main object retains its original size. Best pre-processing before contour detection.



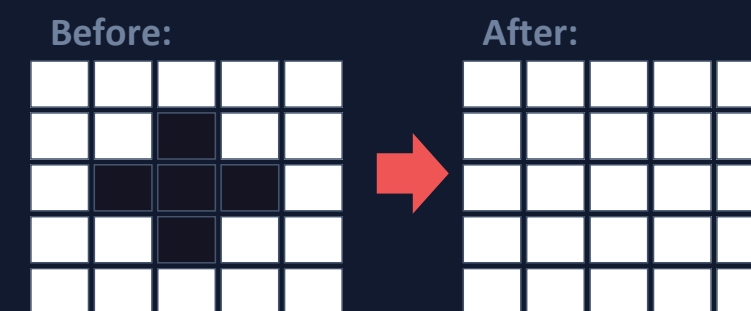
```
cv2.morphologyEx(img, MORPH_OPEN, k)
```

CLOSING

Dilate → Erode

First dilates (fills holes), then erodes (restores size).

Effect: small holes and gaps inside objects are filled while the overall shape stays the same.



```
cv2.morphologyEx(img, MORPH_CLOSE, k)
```


Contour Detection — Finding Shapes

Organized boundary curves you can measure, classify, and reason about

Edges vs Contours — What's the Difference?

Edges are individual disconnected pixels where intensity changes sharply.

They're scattered — no organization, no shape information.

Contours are organized, connected curves that form closed boundaries. Each contour represents one shape that you can measure (area, perimeter, bounding box) and classify (circle, rectangle, triangle).

How findContours Works

Input: a binary image (from thresholding or Canny edges).

OpenCV traces along white pixel boundaries, grouping connected pixels into ordered point sequences — one list per shape.

```
contours, hierarchy = cv2.findContours(  
    binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

Retrieval Modes — What Gets Returned

RETR_EXTERNAL

Most common. Ignores holes and nested shapes inside.

RETR_LIST

Simple collection of every contour, no hierarchy info.

RETR_TREE

Full nesting hierarchy. Know which shape is inside which.

RETR_CCOMP

Objects and their holes. Useful for donut shapes.

Key Contour Properties

- | | | | | | |
|-------------------|---|-----------------------|---|-----------------------|--|
| ■ Area | Size in pixels ² . Filter > 500 to remove noise. | ■ Perimeter | Boundary length. True = closed contour. | ■ Bounding Box | x, y, w, h — axis-aligned rectangle. |
| ■ Centroid | Center of mass (cx, cy). | ■ Shape Approx | 3 verts = △, 4 = □, 8+ ≈ ○. | ■ Convex Hull | Outer envelope. Solidity = area/hull_area. |

5b

Contour Analysis — Measuring & Classifying Shapes

Practical code patterns for extracting information from contours

Shape Classification by Vertex Count

Use `cv2.approxPolyDP()` to simplify a contour to fewer vertices.
The number of vertices tells you what shape it is:
`epsilon = 0.02 * cv2.arcLength(contour, True)` ← controls simplification



Triangle

3 vertices



Rectangle

4 vertices



Pentagon

5 vertices



Circle

8+ vertices

Filtering Contours — Removing Noise

Real images produce hundreds of tiny contours from noise.
Filter by area, aspect ratio, or solidity to keep only real objects:

```
# Filter by minimum area
big = [c for c in contours if cv2.contourArea(c) > 500]
```

```
# Sort by area (largest first)
sorted_c = sorted(contours, key=cv2.contourArea,
                  reverse=True)
```

```
# Aspect ratio filter (near-square objects)
x,y,w,h = cv2.boundingRect(c); ar = w/h
```

Complete Contour Analysis Pipeline

1. Read Image

Load BGR image

2. Gray scale

Convert to gray

3. Blur

GaussianBlur (5,5)

4. Thresh old

Otsu's binary

5. Morph

Opening + Closing

6. Find Contours

RETR_EXTERNAL

7. Filter

Area > 500 px

8. Analyze

Area, bbox, centroid

Feature Detection — Distinctive Keypoints

Repeatable, distinctive points that enable matching, tracking, and recognition

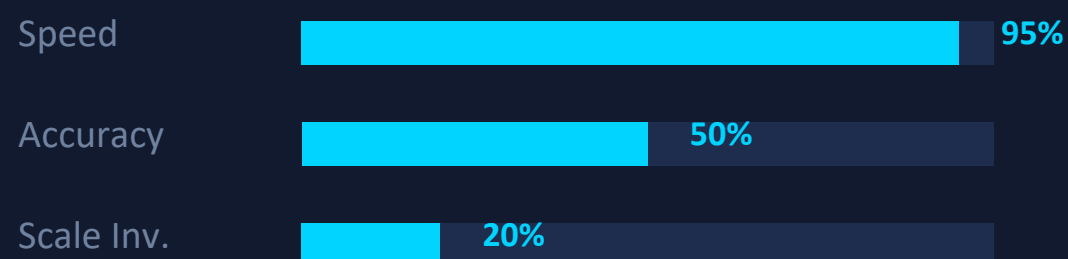
Harris Corners

Corner Detection

- Measures intensity change in all directions
- Flat = no feature, Edge = weak, Corner = strong
- Fast computation, good for real-time
- NOT scale-invariant (misses at different zooms)
- Parameters: blockSize, ksize, k (sensitivity)

```
cv2.cornerHarris(gray, 2, 3, 0.04)
```

Performance:



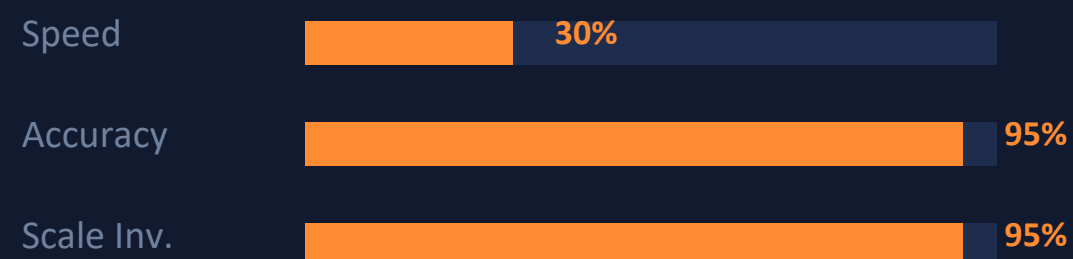
SIFT

Scale-Invariant Feature Transform

- 4 steps: DoG → Localize → Orient → Describe
- Invariant to scale AND rotation changes
- 128-dimensional float descriptor per keypoint
- Most accurate for matching across viewpoints
- Slower — not ideal for real-time use

```
sift = cv2.SIFT_create()
```

Performance:



ORB

Oriented FAST + Rotated BRIEF

- FAST corner detection + BRIEF descriptors
- Rotation invariant, partial scale invariance
- 32-byte binary descriptor (very compact!)
- 10× faster than SIFT, completely free
- Best balance of speed and quality

```
orb = cv2.ORB_create(nfeatures=1000)
```

Performance:



Choose: Harris for simple real-time corners | SIFT for max accuracy | ORB for real-time + good quality (10× faster, free, 32-byte descriptors)

Feature Matching Between Images

Connecting keypoints across different views to find correspondences

How Feature Matching Works

Detect keypoints in both images. Each keypoint gets a descriptor (a numerical fingerprint). Match descriptors between images by finding the most similar pairs.



Two Matching Algorithms

BFMatcher (Brute-Force)

Compares every descriptor in Image A with every descriptor in Image B.

Simple and accurate, but $O(n^2)$ — slow for large feature sets.

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True) # for ORB
```

FLANN (Fast Library for Approx Nearest Neighbors)

Uses KD-trees or LSH to approximate nearest neighbors.

Much faster than brute-force for large datasets.

```
flann = cv2.FlannBasedMatcher(index_params, search_params)
```

Complete Feature Matching Pipeline

1. Detect

Create detector (SIFT or ORB). Find keypoints + descriptors.



2. Match

BFMatcher or FLANN compares all descriptor pairs.



3. Filter

Sort matches by distance. Keep top N or use Lowe's ratio test.



4. Verify

RANSAC to reject outliers. Compute homography for perspective warp.



5. Apply

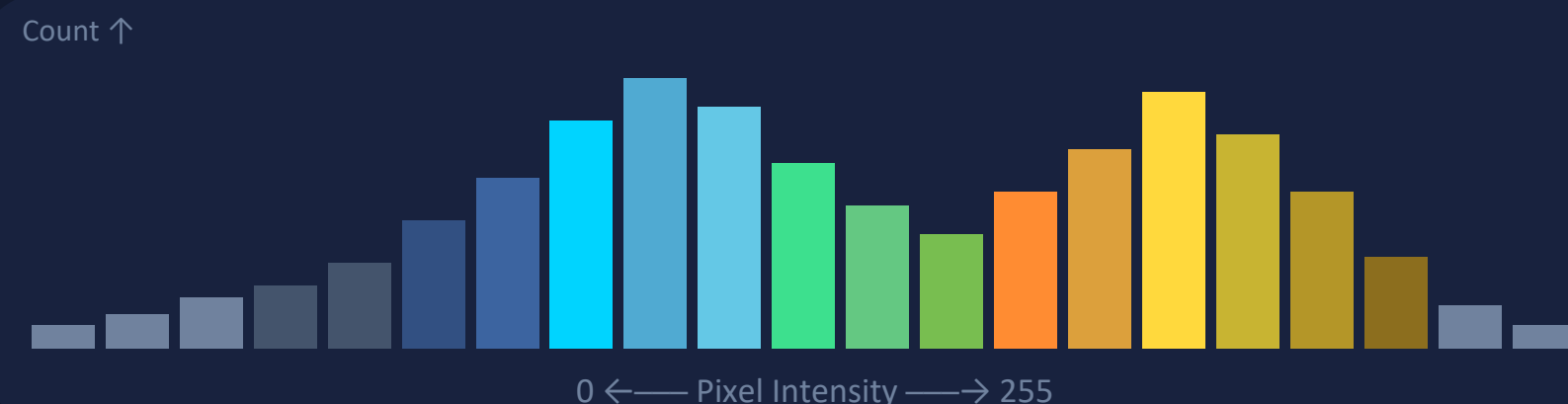
Stitch panoramas, recognize objects, track motion, align images.

8 Histograms — Reading Image Statistics

Understanding pixel intensity distributions for analysis and enhancement

What is an Image Histogram?

A histogram counts how many pixels have each intensity value (0–255). The X-axis is the intensity, Y-axis is the frequency (how many pixels). It tells you about the image's brightness, contrast, and content.



Reading a Histogram

The shape of the histogram instantly tells you about image quality. Use this knowledge to decide if enhancement is needed.

■ Peak clustered LEFT	Underexposed (too dark)	Increase brightness or use equalization.
■ Peak clustered RIGHT	Overexposed (too bright)	Reduce exposure or gamma correction.
■ Narrow peak (center)	Low contrast	Stretch/equalize to use full range.
■ Spread across full range	Good contrast	No enhancement needed — ideal histogram.
■ Two distinct peaks	Bimodal — 2 groups	Object vs background — great for Otsu's.

Histogram Equalization

Stretches the pixel distribution to use the full 0–255 range. Makes dark images brighter and low-contrast images sharper.

Standard equalization applies globally — can over-brighten already bright areas and introduce noise.

Code:

```
equalized = cv2.equalizeHist(gray)
```

CLAHE — Adaptive Equalization

Contrast Limited Adaptive Histogram Equalization. Divides image into tiles, equalizes each tile separately.

clipLimit prevents over-amplification of noise. Much better results than global equalization for real images.

Code:

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
```



Complete Pipeline & Airplane Project

How today's techniques connect into a working analysis system

1	Read Image	<code>cv2.imread('image.jpg')</code>	Load the raw BGR image from disk
2	Grayscale	<code>cv2.cvtColor(img, COLOR_BGR2GRAY)</code>	Reduce to single intensity channel
3	Blur	<code>cv2.GaussianBlur(gray, (5,5), 0)</code>	Smooth noise for cleaner processing
4	Threshold	<code>cv2.threshold(blur, 0, 255, OTSU)</code>	Binary segmentation (foreground/bg)
5	Morphology	<code>morphologyEx(OPEN) + (CLOSE)</code>	Clean noise specks, fill holes
6	Contours	<code>findContours(clean, RETR_EXTERNAL)</code>	Extract and measure all shapes
7	Features	<code>ORB_create() → detectAndCompute()</code>	Find keypoints for matching/tracking



Airplane Turnaround — How Day 2 Applies

Thresholding

Separate aircraft from tarmac background

Canny Edges

Find outlines of aircraft, jet bridges, equipment

Morphology

Clean noisy detections from surveillance footage

Contours

Measure position & size of detected objects

Features

Track specific aircraft parts across video frames

Histograms

Detect lighting/activity changes over time



Day 2 — Key Takeaways

1

Thresholding creates binary images from grayscale

Otsu's finds the best threshold automatically. Adaptive handles shadows. Always blur before thresholding.

2

Gradients measure intensity change — the math behind edges

Sobel computes directional change (X or Y). Magnitude = edge strength. Use CV_64F to keep negatives.

3

Canny is the gold standard 4-step edge detector

Blur → Gradient → Non-max suppression → Hysteresis. Use Auto-Canny for adaptive threshold selection.

4

Morphology cleans binary images before analysis

Opening (erode→dilate) removes noise. Closing (dilate→erode) fills holes. Essential pre-processing step.

5

Contours give you measurable, classifiable shapes

Area, perimeter, bounding box, centroid. Filter by area > 500. Approximate to count vertices for classification.

6

Features enable cross-image matching and tracking

Harris for corners. SIFT for accuracy. ORB for real-time (10× faster, free). BFMatcher or FLANN to match.

TOMORROW — Day 3: Video Analysis & Object Tracking

Frame differencing • Background subtraction • Optical flow • Object tracking algorithms — turning static analysis into dynamic motion detection